

# Taller WAAP con IA/ML, RASP y DevSecOps

## Informe de laboratorio — Fases 0 a 4

Universidad Distrital FJDC — Mecanismos de Seguridad Informática. Prof.  
Octavio J. Salcedo Parra.

Prototipo: OWASP Juice Shop 20.2.0 + ModSecurity CRS 3.3.10 (PL1). Entorno:  
Ubuntu 22.04, Docker + Docker Compose. Acceso vía WAAP:  
<http://localhost:8080> (backend directo :3000).

Rama del trabajo: lab/fases-0-4. Este documento se genera de forma  
reproducible con `build_informe_full.py` (ver Anexo A).

### Fase 0 — Preparación del entorno

**Objetivo:** levantar la infraestructura base (PDF p.9).

Paso a paso ejecutado:

```
cd taller-waap
cp .env.example .env # JUICE_PORT=3000, WAAP_PORT=8080, PARANOIA=1
docker compose pull
docker compose up -d && docker compose ps
curl -i http://localhost:8080 # 200 OK, Server: nginx (proxy WAAP)
```

Resultado: ambos contenedores Up (waap-proxy healthy). Nota: el usuario  
debió añadirse al grupo docker (`sudo usermod -aG docker $USER`) por  
permiso denegado en `/var/run/docker.sock`.

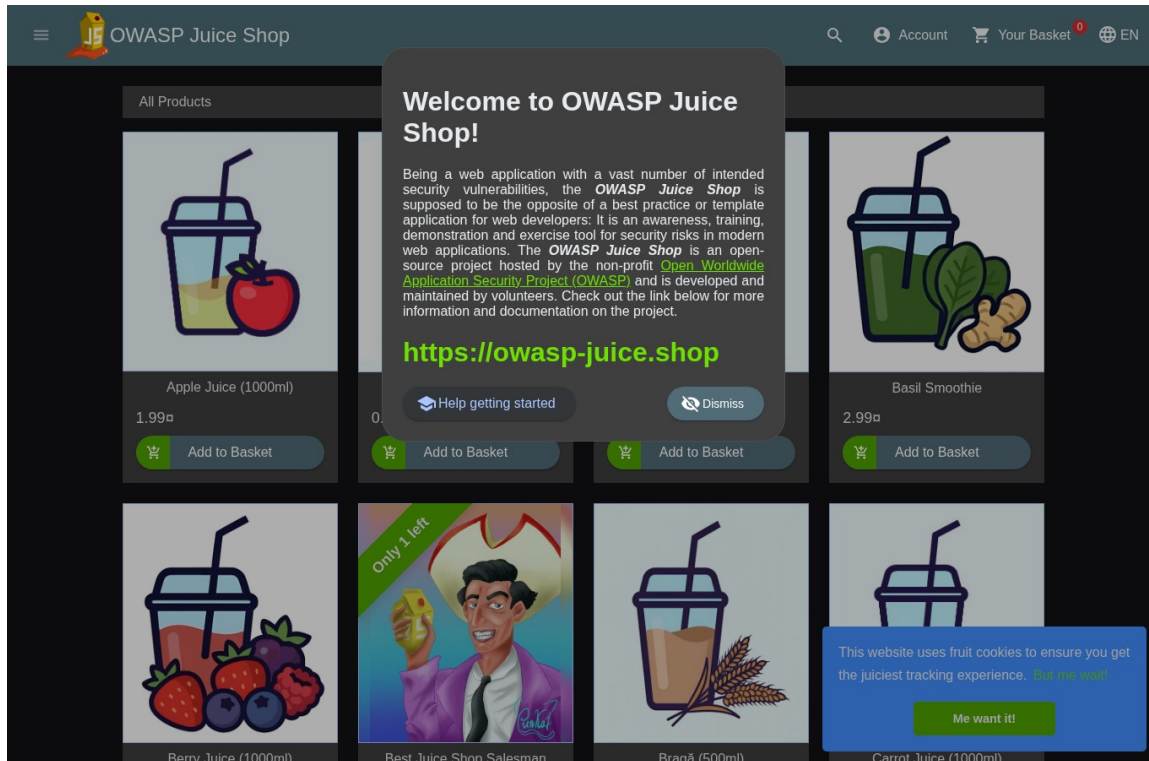
```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED    STATUS    PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ€¢"
juice-shop    32 minutes ago    Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â€¢"
waap-proxy    32 minutes ago    Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
```

```
===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200
```

```
===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1
```

```
===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"
```

**Figura 1. Juice Shop servido a través del proxy WAAP (:8080).**



## Fase 1 — Aplicación objetivo vulnerable

**Objetivo: blanco controlado con vulnerabilidades documentadas (PDF p.10).**

Paso a paso ejecutado:

```
curl -s http://localhost:8080/rest/admin/application-version # {"version":"20.2.0"}
curl -s "http://localhost:8080/rest/products/search?q=apple" # JSON success vía
WAAP
# Navegador: http://localhost:8080/#/score-board (retos por categoría OWASP)
```

Línea base de vulnerabilidades (entregable Fase 1):

| # | Vulnerabilidad            | OWASP Top 10  | Endpoint                 |
|---|---------------------------|---------------|--------------------------|
| 1 | Inyección SQL en login    | A03 Injection | POST<br>/rest/user/login |
| 2 | XSS reflejado en búsqueda | A03 Injection | GET<br>/rest/products/se |

|   |                                |                           |                                   |
|---|--------------------------------|---------------------------|-----------------------------------|
|   |                                |                           | arch?q=                           |
| 3 | IDOR en cesta/pedidos          | A01 Broken Access Control | /rest/basket/:id                  |
| 4 | Exposición de hash de password | A07 / Sensitive Data      | GET /rest/user/whoami, /api/Users |
| 5 | Path traversal / LFI           | A01 / A03                 | GET ...q=../../etc/passwd         |

Figura 2. Score Board (1%, categorías OWASP).

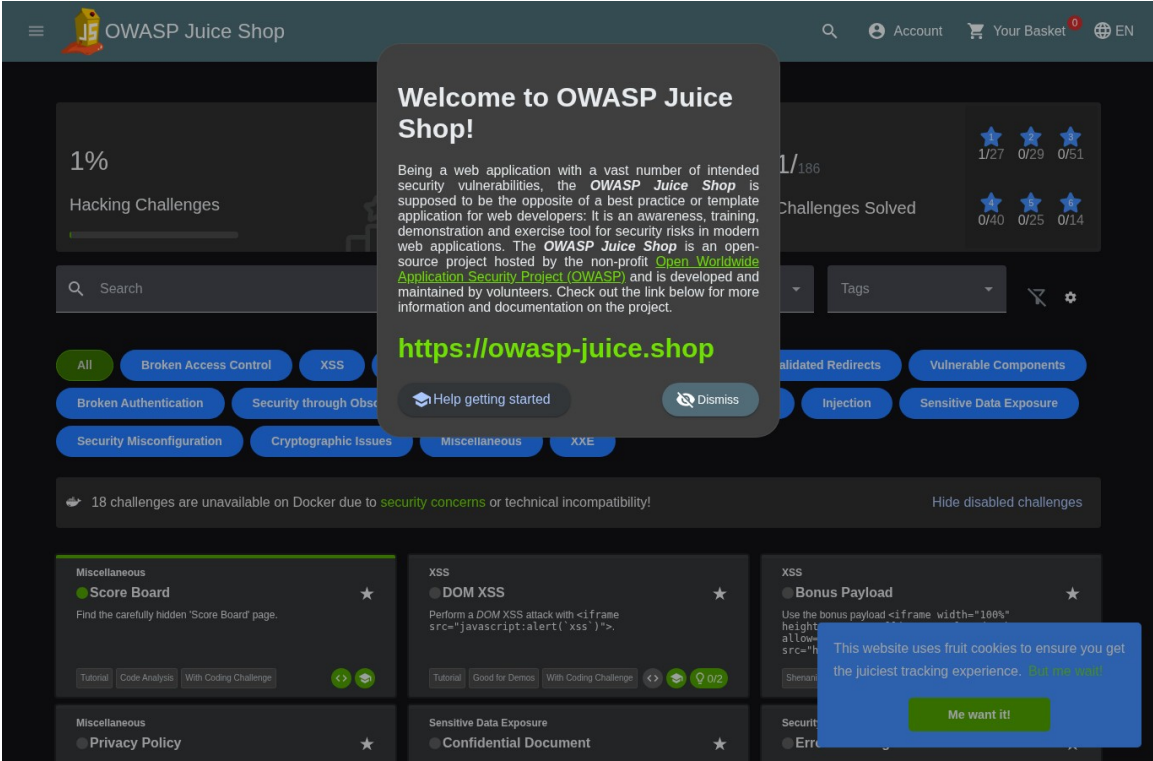


Figura 3. Tráfico normal: búsqueda vía WAAP (200).

Pretty-print ☐

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "Apple Juice (1000ml)",
      "description": "The all-time classic.",
      "price": 1.99,
      "deluxePrice": 0.99,
      "image": "apple_juice.jpg",
      "createdAt": "2026-09-17 23:55:44.453 +00:00",
      "updatedAt": "2026-09-17 23:55:44.453 +00:00",
      "deletedAt": null
    },
    {
      "id": 24,
      "name": "Apple Pomace",
      "description": "Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href=\"#recycle\">sent back to us</a> for recycling.",
      "price": 0.89,
      "deluxePrice": 0.89,
      "image": "apple_pressings.jpg",
      "createdAt": "2026-09-17 23:55:44.462 +00:00",
      "updatedAt": "2026-09-17 23:55:44.462 +00:00",
      "deletedAt": null
    },
    {
      "id": 47,
      "name": "Pineapple Juice (1000ml)",
      "description": "Tropical refreshment from the finest sun-ripened pineapples.",
      "price": 2.99,
      "deluxePrice": 2.99,
      "image": "pineapple_juice.png",
      "createdAt": "2026-09-17 23:55:44.464 +00:00",
      "updatedAt": "2026-09-17 23:55:44.464 +00:00",
      "deletedAt": null
    }
  ]
}
```

## Fase 2 — Capa base WAF/WAAP con CRS (PL1)

**Objetivo: primera línea de defensa con OWASP CRS y evidencia de bloqueo + evasión (PDF p.10).**

Paso a paso ejecutado:

```
# Nivel de paranoia (PDF: iniciar en PL1)
docker compose exec waap-proxy env | grep -E 'PARANOIA|BACKEND'
# PARANOIA=1, BLOCKING_PARANOIA=1, BACKEND=http://juice-shop:3000
# Prueba de detección básica:
curl -i --get --data-urlencode "q=' OR '1'='1' \" \
  http://localhost:8080/rest/products/search # -> 403 Forbidden
# Prueba de evasión (payload fragmentado en 2 parámetros):
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1" # -> 200
# Logs (esta imagen emite el audit log a docker logs, no a
/var/log/modsec_audit.log):
docker compose logs waap-proxy | grep -E '942100|949110'
```

Resultado: SQLi clásico/encoded/comentarios/UNION/XSS/traversal → 403 (regla 942100 libinjection + 949110 anomaly score). Solo el fragmentado q=' & x=OR 1=1 → 200 ({ "status": "success", "data": [] }): el CRS PL1 evalúa cada parámetro por separado.

**Figura 4. Bloqueo CRS: SQLi clásico → 403 Forbidden (nginx).**

## 403 Forbidden

nginx

**Figura 5. Evasión: payload fragmentado → 200 OK (base comparativa para Fase 3).**

Pretty-print ☐

```
{"status": "success", "data": []}
```

## Fase 3 — Módulo de detección basado en IA/ML

**Objetivo: modelo de anomalías entrenado solo con tráfico normal (PDF p.11-12).**

Paso a paso ejecutado:

```
python3 -m pip install --user scikit-learn pandas numpy joblib
# 1. Tráfico normal simulado a ritmo humano (~15 min, 120 reqs vía WAAP):
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py # -> logs/access_normal.jsonl
# 2. Features literales del PDF (url_length, body_length, entropy, n_params,
#     has_suspicious_chars, req_per_minute):
python3 waap/ml/extract_features.py # -> logs/features_normal_traffic.csv
# 3. Entrenamiento literal (IsolationForest 200/0.02 + StandardScaler):
python3 waap/ml/train_model.py # -> waap/ml/model.pkl (en .gitignore)
# 4. Evaluación: matriz 10 + 2 umbrales + ráfaga bot:
python3 waap/ml/evaluate.py
```

Lección metodológica: la primera pasada a ritmo máquina (0.05-0.4 s) contaminó req\_per\_minute (rpm hasta 200 = 'normal'); se repitió a ritmo humano (4-11 s, rpm  $8 \pm 1$ ), equivalente a los 15-20 min de navegación manual que pide el PDF.

Modelo entrenado y guardado en waap/ml/model.pkl (120 muestras)

Matriz de 10 peticiones (entregable Fase 3):

| etiqueta                     | score   | <-0.05  | <0.0    | esperado |
|------------------------------|---------|---------|---------|----------|
| busqueda comun               | 0.0287  | normal  | normal  | normal   |
| login valido                 | -0.1024 | ANOMALO | ANOMALO | normal   |
| home                         | -0.1112 | ANOMALO | ANOMALO | normal   |
| producto                     | -0.0141 | normal  | ANOMALO | normal   |
| score-board                  | -0.0202 | normal  | ANOMALO | normal   |
| SQLi clasico                 | -0.0253 | normal  | ANOMALO | ataque   |
| SQLi encoded (Fase 2)        | -0.0845 | ANOMALO | ANOMALO | ataque   |
| SQLi comentarios             | -0.0428 | normal  | ANOMALO | ataque   |
| evasion fragmentada (Fase 2) | -0.0095 | normal  | ANOMALO | ataque   |
| XSS reflejado                | -0.0759 | ANOMALO | ANOMALO | ataque   |

```
--- Rafaga bot: 20 consultas rapidas (req_per_minute alto) ---
req_per_minute= 5: score=+0.1571 anomalo(<-0.05)=False
req_per_minute= 30: score=+0.1690 anomalo(<-0.05)=False
req_per_minute= 120: score=+0.1690 anomalo(<-0.05)=False
```

Umbrales (detección vs falsos positivos):

```
umbral -0.05: TP=2/5 FN=3/5 FP=2/5 TN=3/5 deteccion=40% FPR=40%
umbral +0.00: TP=5/5 FN=0/5 FP=4/5 TN=1/5 deteccion=100% FPR=80%
```

Discusión (entregable PDF): umbral -0.05 → 2/5 ataques con 2 FP (login válido, home); umbral 0.0 → 5/5 con 4 FP. La ráfaga bot apenas mueve el score (+0.16→+0.17): se depuró hasta la causa (los árboles no ordenan más allá del máximo de entrenamiento), por lo que req\_per\_minute solo no cruza el umbral — limitación documentada y propuesta Q7 (regla híbrida

tasa+score). La evasión fragmentada (q=' aislado) es estadísticamente normal: ni CRS ni ML la ven → motivación de Fase 4.

## Fase 4 — Integración RASP

**Objetivo: protección en tiempo de ejecución con contexto real (PDF p.13).**

Nota metodológica: el rasp\_agent.py Flask del PDF no es integrable a Juice Shop (Node/Express). Se implementó app/rasp-demo/server.js en Express, puerto 1:1 del PDF (mismo SQLI\_PATTERN, query final concatenada, 403 + log). OpenRASP está inactivo desde 2022 sin agente Node vigente; app/rasp\_agent.py queda como referencia literal del PDF.

Paso a paso ejecutado:

```
cd app/rasp-demo && npm init -y && npm i express
RASP_ENABLED=1 RASP_PORT=3001 node server.js & # demo protegido
RASP_ENABLED=0 RASP_PORT=3002 node server.js & # demo vulnerable (control)
# Caso 1: fragmentado directo, sin WAAP (PDF §4.2.1):
curl -X POST :3001/login -d '{"username":"","password":"OR 1=1--"}' # 403
curl -X POST :3002/login -d '{"username":"","password":"OR 1=1--"}' # dump total
# Caso 2: UNION (familia Fase 2): 403 vs dump.
# Caso 3: latencia, 30 reqs alternadas + microbench 200k iters en proceso.
```

Bloqueos:

```
===== CASO 1: fragmentado (user="" pass="OR 1=1--") =====
RASP ON : 403
RASP OFF: {"status":"success","user":"","note":"injection: full dump","rows":2}
===== CASO 2: UNION (familia Fase 2) =====
RASP ON : 403
RASP OFF: {"status":"success","user":"","note":"injection: full dump","rows":2}
===== CASO 2b: ' OR '1'='1 string-form (hueco patron literal PDF) =====
RASP ON : {"status":"success","user":"","note":"injection: full dump","rows":2}
RASP OFF: {"status":"success","user":"","note":"injection: full dump","rows":2}
===== legitimo =====
RASP ON : 200
RASP OFF: 200
```

Logs del agente:

```
RASP: consulta SQL bloqueada en tiempo de ejecucion: SELECT * FROM users WHERE
user='' AND pass='OR 1=1--'
RASP: consulta SQL bloqueada en tiempo de ejecucion: SELECT * FROM users WHERE
user='admin' UNION SELECT * FROM users--' AND pass='x'
```

Lectura: el fragmentado y el UNION confirman la tesis — el contexto de ejecución ve lo que el perímetro no. Hallazgo honesto: la forma string ' OR '1'='1 evade también el regex literal del PDF (solo cubre OR 1=1 numérico); fix propuesto: añadir OR '...'='...'.

## Latencia:

HTTP end-to-end (30 reqs alternadas con warmup, login legitimo):

RASP ON : n=30 media=2.39ms p50=1.41ms p95=8.14ms

RASP OFF: n=30 media=3.70ms p50=1.54ms p95=8.50ms

-> La diferencia ON-vs-OFF es ruido entre procesos (el regex es sub-microsegundo frente a ~2ms de HTTP); ver microbenchmark.

Microbenchmark en proceso (200k iteraciones, mismo patron SQLI\_PATTERN del PDF):

constructor solo : 0.0 micro-s/req

con guard RASP : 0.2 micro-s/req

overhead real del guard: +0.1 micro-s/req

Conclusion: overhead del RASP despreciable (~0.01% de la latencia HTTP). Coherente con el 1-4% reportado por OpenRASP en el peor caso (PDF p.6: limitacion real es el acoplamiento por servicio, no la latencia).

## Matriz comparativa por capas (avance Fase 6)

| Payload / técnica               | Reglas CRS PL1 | IA/ML (umbral 0.0) | RASP                      |
|---------------------------------|----------------|--------------------|---------------------------|
| SQLi clásico                    | BLOQUEA 403    | Detecta            | Bloquea 403               |
| SQLi encoded                    | BLOQUEA 403    | Detecta            | Bloquea (UNION/--/OR 1=1) |
| SQLi fragmentado q=' & x=OR 1=1 | EVADE 200      | No detecta         | Bloquea 403               |
| XSS <script>                    | BLOQUEA 403    | Detecta            | N/A (demo SQL)            |
| ' OR '1'='1 string-form         | BLOQUEA 403    | Detecta (0.0)      | EVADE (hueco patrón)      |
| Ráfaga bot                      | N/A            | No cruza umbral    | N/A                       |

## Anexo A — Paso a paso reproducible (todo lo hecho)

**Secuencia exacta, en orden, desde un clon limpio (rama lab/fases-0-4):**

```
# 0. Requisitos: Ubuntu 22.04, Docker + Compose, Python 3.10, Node 22, LibreOffice.
# Usuario en grupo docker: sudo usermod -aG docker $USER (re-login).
git clone git@github.com:Jhojan98/taller-waap.git && cd taller-waap
git checkout lab/fases-0-4
# FASE 0
cp .env.example .env
docker compose pull && docker compose up -d && docker compose ps
curl -i http://localhost:8080
# FASE 1
curl -s http://localhost:8080/rest/admin/application-version
curl -s 'http://localhost:8080/rest/products/search?q=apple'
# (navegador) http://localhost:8080/#/score-board
# FASE 2
curl -i --get --data-urlencode "q=' OR '1'='1"
http://localhost:8080/rest/products/search
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1"
```

```

docker compose logs waap-proxy | grep -E '942100|949110'
# FASE 3
python3 -m pip install --user scikit-learn pandas numpy joblib
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py
python3 waap/ml/extract_features.py
python3 waap/ml/train_model.py
python3 waap/ml/evaluate.py
# FASE 4
cd app/rasp-demo && npm init -y && npm i express && cd ../..
RASP_ENABLED=1 RASP_PORT=3001 node app/rasp-demo/server.js &
RASP_ENABLED=0 RASP_PORT=3002 node app/rasp-demo/server.js &
curl -X POST localhost:3001/login -H 'Content-Type: application/json' \
  -d "{\"username\":\"'\",\"password\":\"OR 1=1--\"}"
# INFORME (regenerar este documento)
python3 /tmp/opencode/build_informe_full.py # o ruta del generador versionado
soffice --headless --convert-to pdf Informe_Taller_WAAP_Fase0-2.docx

```

Nota: /tmp se limpia al reiniciar; conserve el generador (docs/ o scripts/) si necesita reproducibilidad futura. model.pkl y logs/\*.jsonl se regeneran; no se versionan los .pkl.

## Anexo B — Salidas de terminal (Fases 0-2)

```

===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED        STATUS        PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ(
juice-shop    32 minutes ago    Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â(
waap-proxy    32 minutes ago    Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp

===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200

===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1

===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"

```